

Programming Assignment 4 Report

Student Name: Colton Zachary

UCO Email: *czachary@gmail.com*

Course: CMSC

Assignment: Programming Assignment 4 – Binary Search Tree

Server Username: xl025

1. Overview

The goal of this assignment was to implement a Binary Search Tree (BST) using C++. The BST supports the basic operations of insertion, deletion, searching, and traversal. The program demonstrates how data can be organized in a hierarchical structure that allows efficient searching and manipulation of elements.

The project was implemented using a set of classes including:

- BinaryTreeNode
- BinaryTree
- BSTree

The BSTree class extends the binary tree functionality by enforcing the binary search tree property, which ensures that:

- All values in the left subtree are smaller than the node
- All values in the right subtree are larger than the node

2. Implementation Details

Binary Tree Node

Each node in the tree contains:

- A data value
- A pointer to the left child
- A pointer to the right child

This structure allows the tree to dynamically grow as new nodes are inserted.

Insert Operation

The insert function places a new value in the correct position according to BST rules.

Steps used in the insert algorithm:

1. If the tree is empty, the new node becomes the root.
2. Compare the new value with the current node.
3. If the value is smaller, move to the left subtree.
4. If the value is larger, move to the right subtree.
5. Continue recursively until an empty position is found.
6. Insert the new node at that position.

This ensures the tree maintains the BST property.

Remove Operation

The remove function deletes a node while maintaining the structure of the BST.

There are three possible cases:

Case 1 – Node has no children

The node is simply deleted.

Case 2 – Node has one child

The parent pointer is updated to bypass the deleted node and connect directly to the child.

Case 3 – Node has two children

The node is replaced with its in-order successor, which is the smallest value in the right subtree. The successor is then removed from its original position.

This approach keeps the BST valid after deletion.

Search Operation

The search function locates a value in the tree by comparing the target value with each node encountered.

Steps:

1. Start at the root.
2. If the value matches, return the node.

3. If the value is smaller, search the left subtree.
4. If the value is larger, search the right subtree.
5. Continue until the value is found or a null pointer is reached.

Because of the BST structure, this operation is efficient.

3. Tree Traversals

Two traversal methods were implemented to verify the correctness of the tree operations.

In-Order Traversal

This traversal visits nodes in the following order:

Left → Root → Right

For a BST, this produces the values in sorted order.

Level-Order Traversal

Level-order traversal visits nodes level by level starting from the root.

This traversal demonstrates the structural layout of the tree.

4. Program Testing

The program was tested using two trees.

Tree 1 – Integer Tree

Initial values inserted:

50, 30, 70, 20, 40, 60, 80, 10, 5, 7, 79

Initial traversal results showed that the tree was constructed correctly.

Next, the values 5, 10, and 30 were removed from the tree.

After removal:

- The in-order traversal confirmed the tree still maintained sorted order.
- The level-order traversal showed the correct restructuring of nodes.

The search function was also tested by locating the node containing the value 7, and the program successfully printed the memory address and stored data.

Tree 2 – Character Tree

The following characters were inserted:

D, B, A, C, F, E, Z

The program displayed both in-order and level-order traversals to verify the tree structure.

Next, the values F, C, and D were removed.

After deletion, the traversals confirmed the BST structure remained valid.

The search function successfully located the node containing Z.

5. Additional Testing

An additional integer tree was created with values 0 through 19.

All values were inserted sequentially, then removed one by one to ensure the remove function worked correctly for multiple cases.

After removing all elements, the traversal functions correctly displayed an empty tree.

6. Conclusion

This assignment demonstrated the implementation of a Binary Search Tree and the importance of maintaining the BST property during insertion and deletion operations.

The program successfully implemented:

- Node insertion
- Node removal
- Searching
- In-order traversal
- Level-order traversal

Testing confirmed that the tree structure remained valid after multiple insertions and deletions.

Binary search trees provide an efficient way to organize and retrieve data and are commonly used in many computer science applications such as database indexing and search algorithms.

```
(base) coltonzachary@Coltons-MacBook-Air-4 A4 % ./a.out
Tree 1:
Tree 1 Initial In Order Traversal:
57102030405060707980
Tree 1 Initial Level Order Traversal:
50307020406080107957
Removing 5, 10, and 30 from Tree 1
Tree 1 After Removal In Order Traversal:
720405060707980
Tree 1 After Removal Level Order Traversal:
504070206080779
Address of node containing 7: 0x7c8c00aa0 data from node containing 7: 7

Tree 2:
Tree 2 Initial In Order Traversal:
ABCDEFZ
Tree 2 Initial Level Order Traversal:
DBFACEZ
Removing F, C, and D from Tree 2
Tree 2 After Removal In Order Traversal:
ABEZ
Tree 2 After Removal Level Order Traversal:
EBZA
Data from node containing Z: Z

Int BSTREE
In order!
012345678910111213141516171819
Level order!
012345678910111213141516171819
Removing all items from the tree
In order after removal!

Level order after removal!

(base) coltonzachary@Coltons-MacBook-Air-4 A4 %
```